
LABORATORY EXERCISE #06
Bubble and Selection Sort Implementation

Input: Standard
Output: Standard

PROBLEM DESCRIPTION

Ari is organizing a collection of integers for a data analysis program. Before the numbers can be processed, they must be arranged in ascending order. Ari wants to compare two sorting algorithms: Bubble Sort and Selection Sort.

Your task is to implement both sorting algorithms in a single C++ source file. Given a list of integers, sort the list using Bubble Sort and Selection Sort separately, then display the sorted results to verify that both algorithms produce the same output.

INPUT FORMAT

The first line contains an integer *n*, representing the number of elements in the list.
The second line contains *n* space-separated integers.

CONSTRAINTS

- $1 \leq n \leq 100$
- $-1{,}000 \leq a_i \leq 1{,}000$
- *The input may contain duplicate values.*
- *Both sorting algorithms must arrange the elements in ascending order.*

OUTPUT FORMAT

Display the sorted list produced by each algorithm using the following format:

- The first line must begin with " `Bubble Sort:` followed by the sorted integers separated by spaces. "
- The second line must begin with " `Selection Sort:` followed by the sorted integers separated by spaces. "

PROGRAM SPECIFICATIONS

1. The program must be implemented in one C++ source file only.
2. Implement Bubble Sort and Selection Sort as two separate functions within the same file.
3. Both functions must accept an integer array and its size as parameters.
4. The original input array must be preserved so that both algorithms sort the same initial data. You may use a second array or copy the original array before sorting.
5. Do not use built-in sorting functions from the C++ Standard Library.

6. The `main()` function must handle input, call both sorting functions, and display their respective results.
7. You may use loops, conditional statements, and temporary variables to implement the sorting operations.

SAMPLE INPUT	SAMPLE OUTPUT
`6` `29 10 14 37 13 5`	`Bubble Sort: 5 10 13 14 29 37` `Selection Sort: 5 10 13 14 29 37`

DECLARATION OF AI USAGE / OTHER INTERNET SOURCES

In your submitted solutions, comment the lines/blocks of code generated by AI or sourced from the internet.

SOURCE CODE: *Paste your source code written in C++ here.*

IMPLEMENTATION OF CODE: *Provide a screenshot of a successful compilation of the provided solution, using any IDE.*

GUIDE QUESTIONS

Instructions: Answer the following questions based on your C++ implementation of the Talking to Aliens problem. Support your answers with examples or code explanations when necessary. (5 pts ea)

Instructions: Answer the following questions based on your C++ implementation of Bubble Sort and Selection Sort. Support your answers with examples or code explanations when necessary. **(5 points each)**

---

1. In Bubble Sort, why does the algorithm compare adjacent elements and swap them when they are in the wrong order? Explain how this process moves the largest unsorted element toward its correct position during each pass.
2. In Selection Sort, what is the purpose of the `min` variable and the inner loop? Explain how the algorithm finds the smallest element in the unsorted portion and places it in its correct position. How does this process differ from Bubble Sort?
3. Sorting in ascending order through selection sort, give the arrangement of the following list after the first pass: 30 43 7 15 22 (Explain your steps)
4. Sorting in ascending order through bubble sort, give the arrangement of the following list after the 6th pass: 9 11 24 33 5 9 23 6 1. How many adjacent comparisons does the inner loop perform during the 6th pass, assuming the outer loop starts with i = 0? (Explain your steps)

SUBMISSION OUTPUT

- Submission until **12:00nn, October 26 only.** I will **NOT** entertain late submissions.
- You may use the lab for the completion of this activity.
- **SUBMIT A DOCS COPY OF THIS WITH THE PASTED CONTENT FOR SOURCE CODE AND IMPLEMENTATION ON THE TABLE ABOVE.**
- **Entire source code generated by AI is equivalent to a score of 0.**